

Lecture 5 Design Issues

Outline

- Design Techniques
- Attributes vs Entity
- Entity set vs Relationship set
- Weak entity sets
- Redundancies

Design Techniques

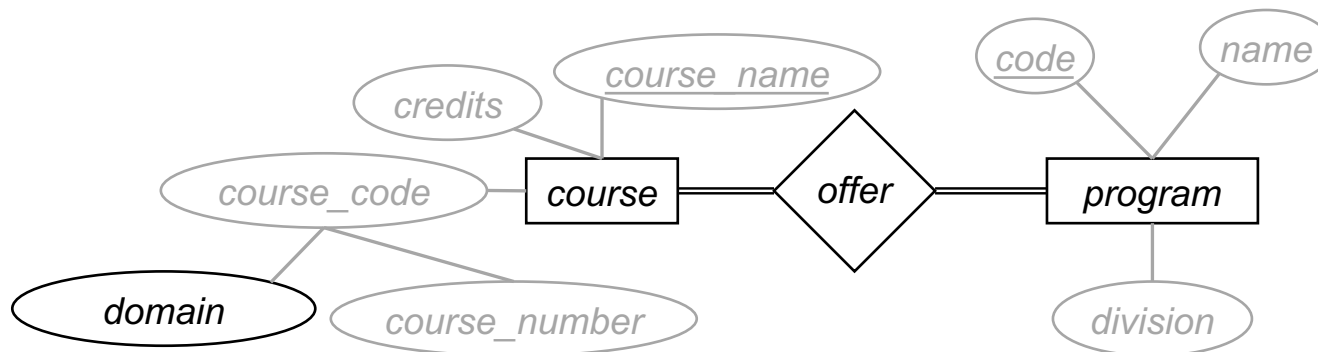
1. Express all constraints (you can express!).
2. Use and do not change terminology and class structure of the application domain.
3. Keep it simple.
 1. Avoid defining entity types that do not serve any purpose.
 2. Don't use an entity set when an attribute will do. Choose an entity set if it helps expressing constraints; otherwise, use an attribute.
4. Avoid redundancy (but derived attributes are okay)!
5. Limit the use of weak entity sets.

Design Techniques

- There are also some decisions that you need to make during the design.
 - Attribute vs Entity
 - Entity set vs Relationship set
 - Strong entity sets vs Weak entity sets
 - Binary relationship vs multi-ary relationship (which has been discussed in Lecture 3)
 - Try to eliminate redundancy
- In most cases, the answer is very subjective. It depends on the real-world problem.
- Sometimes, the answers can be very different even on the same problem but with different assumptions.

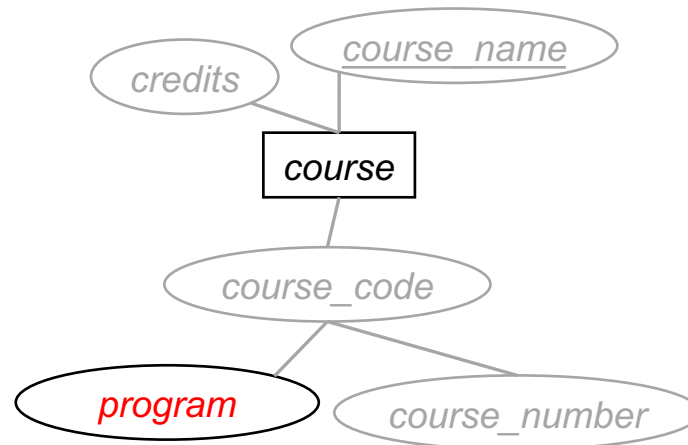
Attribute vs Entity

- Let's go back and look at the entity set “course” again.
- It has an attribute “domain” to show the domain of each course.
- For example, *COMP3013* is in the domain of computer science.



Attribute vs Entity

- Is it possible to use “domain” to represent the program which offers the course?
- If yes, then there is no need for the relationship between “course” and program.
- In consequence, the entity set “program” is not even needed.



Attribute vs Entity

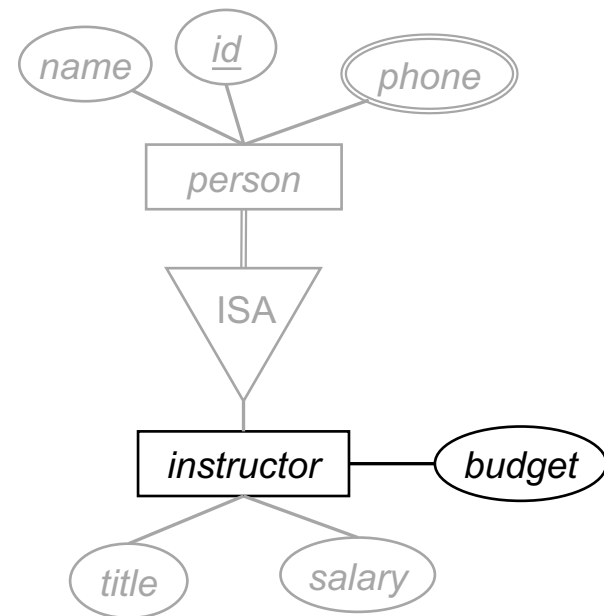
- However, the answer is not that simple. It depends on the real case.
- Both ER models have advantages and disadvantages.
- The model without the relationship set “offer”:
 - the model is simpler and easy to be implemented;
 - but it cannot express the case that a course is offered by multiple programs.
- The model with the relationship set “offer”:
 - it is suitable if some courses are offered by multiple programs;
 - but it has one more relationship set.
- In UIC, the second case is applied. For example, COMP3013 is offered by both CST and DS. Thus, we keep the relationship set “offer” and the entity set “program”.

Entity set vs Relationship set

- In the previous ER diagram, we have also modeled “some instructors are PDs(program director) managing some program”.
- Assume that we want to further model the budget held by every program director.

First attempt:

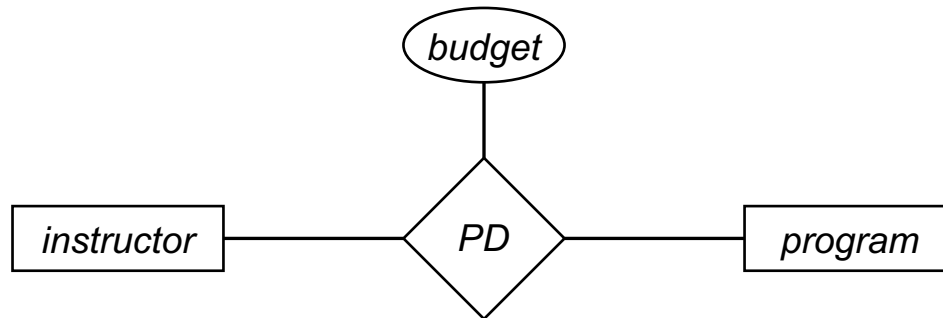
- Add “budget” as an attribute to “instructor”.
- The problem is most instructors are not program directors. And thus, they do not have a budget.



Entity set vs Relationship set

Second attempt:

- Add “budget” as an attribute to the relationship set “PD”.

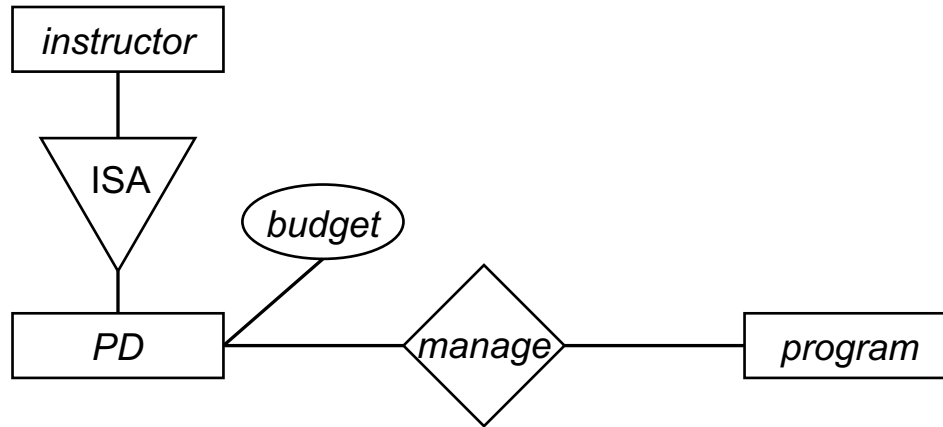


- This model does not work if one instructor can be the program director for more than one programs.
- This model is also misleading. The attribute “budget” in fact belongs to some instructors instead of programs or relationships.

Entity set vs Relationship set

Third attempt:

- Specialize “instructor”.



- This model solves all the problems above.

Weak Entity Sets

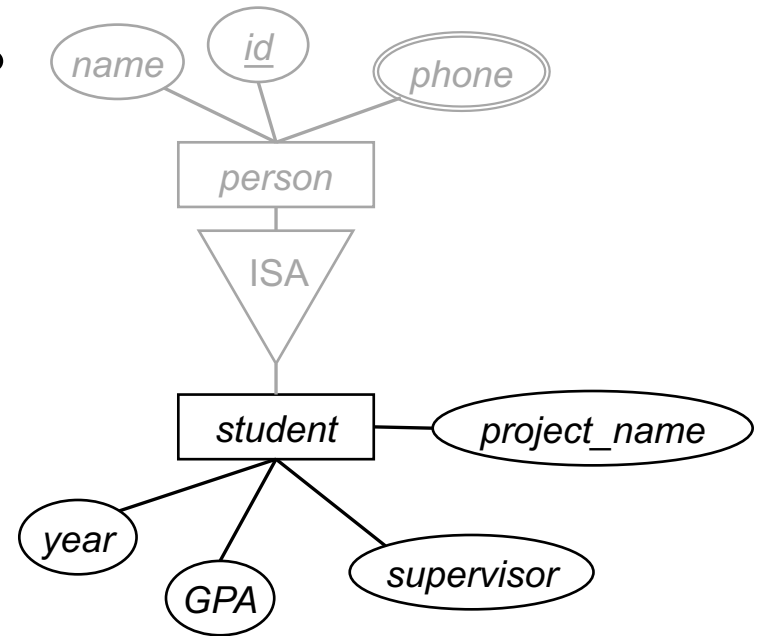
- Database design beginners often doubt that any attribute could be a key.
- Once there is no solution for a key, they make all entity sets weak, supported by all other entity sets to which they are linked.
- In fact, we can artificially create unique IDs for entity sets.
- We use weak entity sets **only if** there is no capability of creating unique IDs.

Redundancy Elimination

- Suppose we want to model that “each student is doing a group project with a supervisor”.
- What is the problem of this design?
- Remember, this is a group project. Multiple students can work on the same project.
- It is possible that

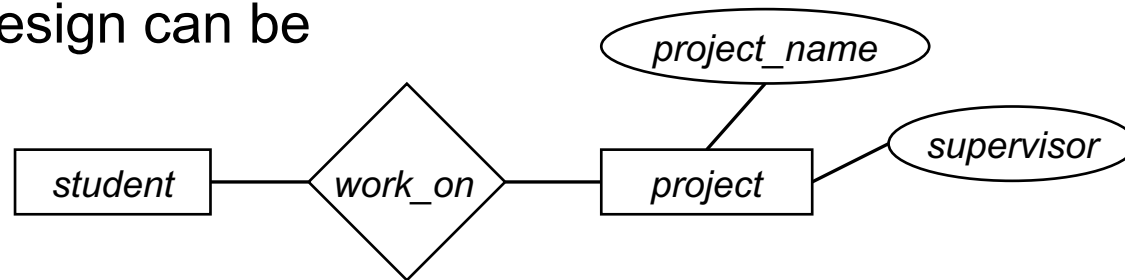
students = {
 (..., Alice, 3D Modeling, Amy),
 (..., Bob, 3D Modeling, Amy),
 (..., Car, 3D Modeling, Amy),
 ... } student project supervisor

- It creates **redundancies**.



Redundancy Elimination

- A better design can be

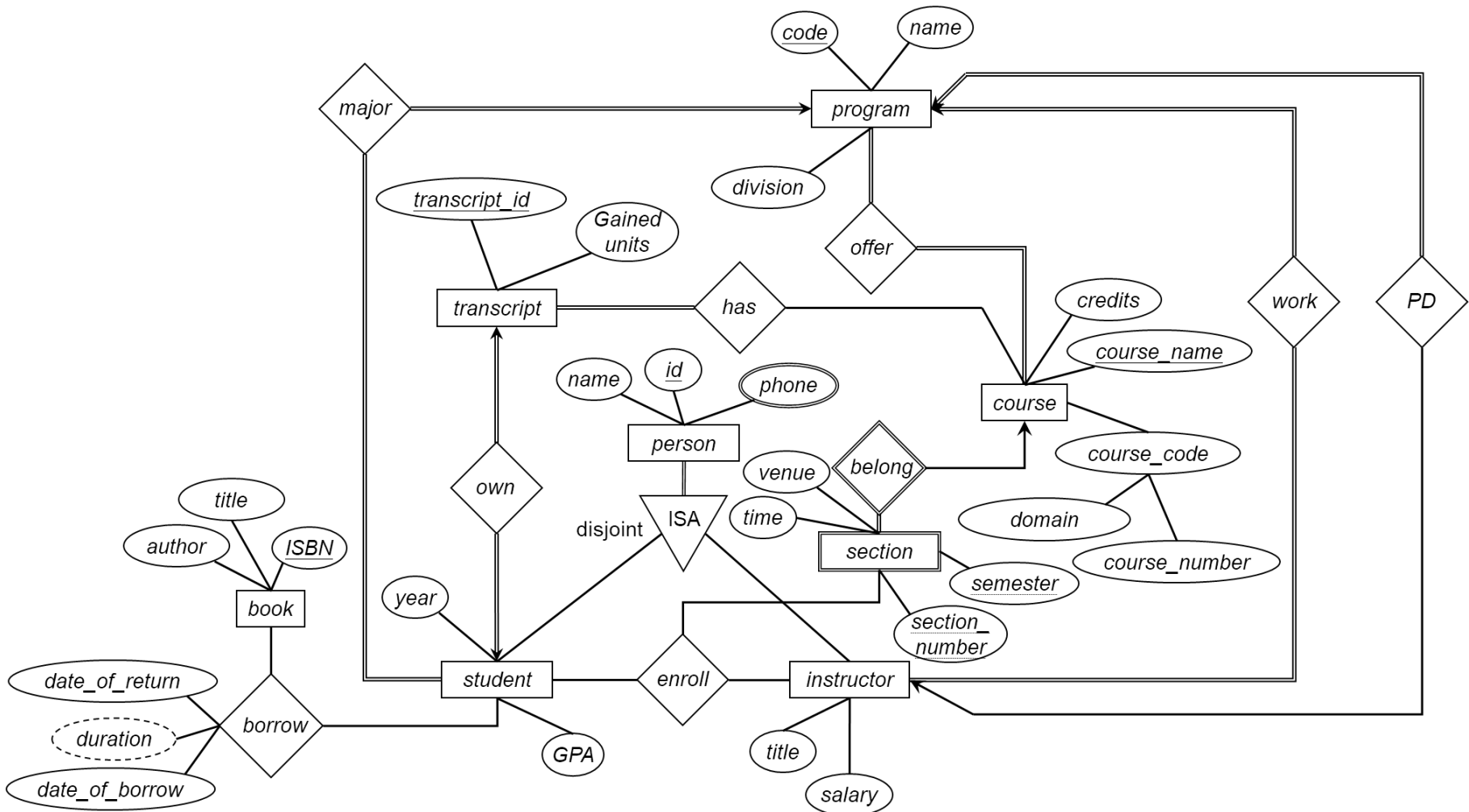


- Because there is a hidden constraint “every project has exactly one supervisor”.
- This type of constraints and optimizations will be discussed in detail in the last chapter.

Exercises

- Consider the UIC database that we have been discussed for previous lectures. (The ER diagram is given on the next page.)
 - Why the entity set *transcript* is useless?
 - What if a student retake some courses?
- Please try to improve the design.

Exercises



Exercises

- Consider a university database for the scheduling of classrooms for final exams. This database could be modeled as the following entity sets:
 - *exam* with attributes *exam_id* and *time*;
 - *course* with attributes *name*, *department* and *c_number*;
 - *section* with attributes *s_number* and *enrollment*;
 - *section* as a weak entity set on *course*; and
 - *room* with attributes *r_number*, *capacity*, and *building*.
 - Different sections of a course will take the exam at the same time, but in different rooms.
- Draw an ER diagram to model the database.

Exercises

- Draw an E/R diagram to model project groups in CS3030. Keep in mind that
 - each enrolled student (identified by a SID) can work at most in one group;
 - each project, identified uniquely by its name, can have multiple groups working on it;
 - identify all the appropriate multiplicity and referential integrity constraints in the diagram; and
 - indicate key attributes in each entity set.

End of Lecture 5